

CSCI-UA 102 — Quiz 8

Binary Search Trees

Name: _____ NetID: _____

Write a method for `BinarySearchTree<V>` that returns the value of the entry with the **third-smallest key**. Assume the tree has size at least 3. *Hint*: an in-order traversal of a BST visits keys in sorted order.

```
private V thirdSmallest() {  
    int[] count = new int[]{0};  
    return helper(root(), count);  
}  
  
private V helper(Position<Entry<Integer,V>> p, int[] count)
```

Write the helper method. You may assume these methods:

```
Position<E> root();  
Position<E> left(Position<E> p);  
Position<E> right(Position<E> p);
```

```
boolean isExternal(Position<E> p);  
boolean isInternal(Position<E> p);  
E getElement(); // on Position
```

Solution

The key idea: an **in-order traversal** of a BST visits keys in sorted order. So the third node visited has the third-smallest key. We use `int[] count` as a shared mutable counter across all recursive calls.

```
private V thirdSmallest() {
    int[] count = {0};
    return helper(root(), count);
}

private V helper(Position<Entry<Integer,V>> p, int[] count) {
    if (isExternal(p)) return null;

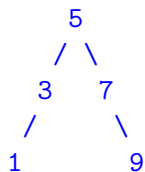
    V left = helper(left(p), count);           // recurse left
    if (left != null) return left;             // found in left subtree

    count[0]++;                                // visit current node
    if (count[0] == 3)
        return p.getElement().getValue();     // this is the 3rd

    return helper(right(p), count);           // recurse right
}
```

Why `int[]` instead of `int`? Java passes primitives by value. If we used `int count`, each recursive call would get its own copy and increments would not be seen by the caller. Wrapping it in `int[]` gives us a shared, mutable counter: when any call does `count[0]++`, every other call holding the same array reference sees the update.

Walk-through: BST with keys [1, 3, 5, 7, 9]:



1. `helper(5, {0})` → recurse left
2. `helper(3, {0})` → recurse left
3. `helper(1, {0})` → left is external, returns null
4. Visit 1: `count` = 1. Not 3. Recurse right → external, returns null.
5. Back at 3: left returned null. Visit 3: `count` = 2. Not 3. Recurse right → external, returns null.
6. Back at 5: left returned null. Visit 5: `count` = 3 → return value at key 5. Done!